Where Systems Meet Reality

And what happens when they don't agree

A short case deck showing how I removed real-world friction across logistics, finance, and other systems.
Not a CV. Six real patterns, six real systems, and what changed after.

Time-sensitive Synchronization • Data Transformation Pipelines • External Dependency Orchestration
Legacy Modernization • Constraint-aware Navigation • Physical-to-Digital Feedback Loops

I stabilize systems under real-world pressure

- ... where data stops flowing
- ... where systems drift out of sync
- ... where manual work starts appearing
- ... where assumptions quietly break
- ... where reality disagrees

Marshall Laszlo Toth

Designing systems that don't snap under real-world pressure
Focused on the points where data stops flowing and manual work begins.

Time-sensitive Synchronization

Reality changes while the user is still clicking.

In systems where availability, stock, or physical state changes continuously, timing becomes a critical failure point. A user, a partner, and the system itself all operate on slightly different timelines. Without proper synchronization, this gap turns into overbooking, stock conflicts, or irreversible data loss.

Across hospitality, logistics, and manufacturing, the core problem was consistent: decisions were made on stale or incomplete information, forcing manual intervention. The goal was not perfect real-time accuracy, but designing layers that could tolerate delay, absorb inconsistency, and still produce valid outcomes.

Risskov - Last-minute hotel bookings

Availability and pricing changed while customers were browsing offers
Manual approval by call center created a constant bottleneck
Hotels had little incentive to keep data up to date
Introduced self-managed calendars + pricing interface for partners

Risskov - Synchronization layer

Designed booking flow that tolerates stale availability
Built auto-approval logic for consistent cases
Fallback path for conflicts with alternative offers
Shifted system from reactive approval to proactive validation

WebShippy - Warehouse fulfillment

Orders, stock, and physical picking constantly drifted out of sync
Reservation of stock under real-time order pressure was fragile
Barcode labels, printers, and human flow caused unexpected delays
Aligned digital orders with physical movement using unified tooling

WebShippy - On-floor reality

Shared printers across packing stations to remove bottlenecks
Android barcode scanners introduced for real-time tracking
Iterated physical materials to prevent label loss
System allowed locating any item within ~40 cm storage precision

Furniture manufacturing - Order lifecycle

Customers could modify orders while production was about to start
Sales and factory operated on different timing assumptions
Manual database edits introduced inconsistencies
Introduced controlled delay + approval flow between sales and production

Outcomes:

80% of hotel bookings auto-approved without human intervention.
Warehouse operations became traceable in real time, with precise item location.
Manual synchronization work significantly reduced across all systems.

Designed systems that remain stable even when time, data, and reality are slightly out of sync.

References:
Laravel, Vue, GraphQL, custom warehouse tooling
Multi-country booking platform, 1000+ daily orders environment

Data Transformation Pipelines

Every system has its own version of truth. None of them fully agree.

Modern systems rarely operate in isolation. Data flows between platforms with different structures, assumptions, and levels of reliability. What looks like a simple import is often a negotiation between incompatible models, partial information, and inconsistent quality.

Before proper transformation layers, this friction shows up as broken workflows, manual reconciliation, and silent corruption. The goal was not just to move data, but to reshape it into something consistent, traceable, and trustworthy enough for downstream decisions.

Ordio - Workday HR integration

Incoming HR data arrived inconsistent, incomplete, and sometimes incorrect
Source system was unresponsive, making validation difficult
Downstream workflows broke on edge cases and missing fields
Introduced structured transformation and validation between systems

Ordio - Migration pressure

Client onboarding depended on clean data imports
Mismatch between Workday structure and internal domain model
Handled partial data while keeping workflows usable
Resulted in smoother migrations with fewer manual fixes

TechTeamer - Digital lending pipeline

Loan decisions depended on multiple external data sources
Identity, risk scoring, and financial data arrived asynchronously
Partial or delayed data blocked or corrupted loan lifecycle
Built flow where loosely coupled services converge into a final decision

E-commerce & logistics - Multi-platform ingestion

WooCommerce, Shopify and others all structured orders differently
Needed one unified model for fulfillment and shipping
Transformed heterogeneous inputs into consistent internal format
Enabled scalable order handling across 100+ partners

Manufacturing - From design to materials

Architectural designs had to become raw material requirements
Cross-domain translation between design, production, and logistics
Required cooperation across non-technical and technical roles
Turned abstract configurations into actionable production data

Outcomes:

Reliable data flows replaced manual reconciliation across multiple domains.
Client migrations became smoother, loan decisions faster, and order processing scalable across 1000+ daily transactions.

Built pipelines that turn inconsistent, multi-source data into something stable enough to act on.

References:
PHP, Laravel, Symfony, MySQL
Workday integration, fintech microservices, multi-platform e-commerce

External Dependency Orchestration

The outside world does not respect your internal architecture.

Many business-critical systems depend on APIs, partners, and external services that can change behavior without warning. One unstable dependency is manageable. Several, interacting across the same workflow, quickly become a source of cascading errors, hidden state corruption, and support overhead.

The real challenge was not simply connecting systems, but protecting the core domain from volatility outside your control. That meant isolating failures, making dependency behavior visible, and keeping the internal flow reliable even when external input was late, incomplete, or inconsistent.

Ordio - Workday imports

Source data arrived with inconsistencies and missing structure
Workday responses were not something you could fully trust as-is
Downstream logic broke when edge cases were not isolated
Built a boundary that contained external unpredictability

Risskov - Hotel partner systems

Availability and pricing came from many external partner sources
Partner behavior varied across countries and business rules
Manual coordination was often needed just to keep things usable
Added orchestration so partner volatility stayed outside the core

TechTeamer - Identity verification

Remote identification depended on a third-party service
Loan flow could not safely proceed on partial trust alone
External verification had to fit into a regulated decision chain
Structured the dependency so the internal state remained auditable

Diligent - Enterprise services

Large platform changes required coordination with surrounding services
Cloud and TypeScript transitions added more moving parts
Observability mattered as much as the integration itself
Kept the system stable while the dependency surface evolved

Outcomes:

Reduced the risk of cascading failures and silent corruption.
External integrations became more predictable for the business, and internal workflows stayed usable even when partner systems misbehaved.

Built the buffer between unstable external systems and the core domain that had to keep working.

References:
API boundaries, validation, logging, recovery paths
Workday, hotel partners, identity verification, enterprise services

Legacy Modernization

The hardest upgrade is the one nobody is allowed to notice.

Long-lived systems do not give you a clean restart window. They carry audit trails, business rules, old assumptions, and people who still depend on them every day. Changing them is less like rewriting software and more like replacing parts of a machine while it is still running.

The work here was to modernize gradually without breaking trust, continuity, or compliance. That meant improving the structure underneath while keeping the visible behavior stable enough that users, auditors, and business operations could continue without disruption.

Diligent - Legacy PHP system

Core business system could not be stopped or reset
Auditability and compliance had to remain intact
Training on AWS and TypeScript made sense only in context
Modernization had to respect the existing system of record

NextTuesday - Shrinking team, live clients

Department restructuring reduced staffing and context continuity
Knowledge gap became part of the operational risk
Systems still had to support active clients and deliveries
Incremental maintenance kept the platform alive during decline

Furniture company - Manual queue disappears

Factory office once had a daily queue of workers entering data
Shared spreadsheet workflow was the bottleneck
Over time the wasteful manual step vanished
Modernized flow quietly removed friction without dramatic breakage

TechTeamer - Early microservice transition

Regulated lending required safer boundaries over time
System of record had to survive distributed changes
Partial understanding was normal during evolution
Built confidence in gradually changing a critical platform

Outcomes:

Modernization happened without breaking trust, compliance, or continuity. The visible result was less manual work, better maintainability, and systems that could keep evolving instead of freezing in place.

Modernized critical systems by changing the internals without disturbing the surface.

References:
PHP, Laravel, AWS, TypeScript, custom frameworks
Audit trails, compliance, incremental change

Constraint-aware Navigation

Some interfaces are really maps through impossible terrain.

When a product or workflow contains too many valid combinations, the challenge is no longer display. It is guidance. Users need to move through a space of rules, exceptions, and dependencies without getting trapped in invalid states or overwhelmed by the number of choices.

This pattern showed up early in furniture configuration and later in operational workflows for non-technical users. The goal was to make complex decision spaces feel manageable by introducing structure, validation, and progressive movement instead of exposing raw complexity all at once.

Furniture configurator

Product combinations grew to an extreme scale
Users could easily create invalid or impossible configurations
The domain felt more like a navigation problem than a form
Built logic that guided choices instead of enumerating everything

Sales to factory handoff

Order changes were allowed only within specific timing rules
After production started, changes needed factory approval
The system had to reflect operational constraints clearly
Reduced confusion by making the rules visible in the flow

Ordio - Workforce workflows

Deskless users needed safe step-by-step guidance
Backend complexity had to stay hidden from the operator
Invalid states had to be blocked before they spread
Used process-oriented UI to reduce mistakes

Approval and onboarding flows

Non-technical users were often working under time pressure
Decision paths changed depending on data and context
Hard stops needed to feel like help, not punishment
Made the system act like a guide instead of a trap

Outcomes:

Users created fewer invalid states and needed less manual intervention.
Complex products and workflows became easier to complete, approve, and maintain at scale.

Turned combinatorial complexity into a path people could actually walk through.

References:
React, Vue, TypeScript, HTML, CSS
Configurators, onboarding, approvals, operational flows

Physical-to-Digital Feedback Loops

Reality does not wait for the database to catch up.

In physical systems, the source of truth is not the screen. It is the warehouse floor, the excavation site, the sensor, or the object in someone's hands. That makes timing, traceability, and immediate capture much more important than elegance.
This pattern appears wherever manual work, hardware, and formal records must stay aligned. The friction is usually visible first as lost context, delayed updates, paper shuffling, or small errors that become expensive once the physical world has moved on.

WebShippy - Warehouse operations

Digital orders had to match physical picking and packing
Shared printers, handheld scanners, and label flow all mattered
Friction showed up as delays, lost stickers, and stock confusion
Aligned tools with the pace of the warehouse floor

Archbau - Archaeological documentation

Once a layer is removed, context is gone forever
Documentation had to happen before construction destroyed evidence
Manual methods were still dominant on-site
Treated excavation as irreversible data capture

Future Property Trade - Production and fulfillment

Digital commerce had to become real merchandise and shipping
Demand, manufacturing, and logistics were tightly coupled
Mismatch between intent and reality created downstream strain
Helped keep the loop tight from order to delivery

ESP32, RFID, sensors, field experiments

Hardware taught immediate limits in power, noise, and timing
Some experiments failed usefully rather than cleanly
Observability became more important than perfection
Built intuition for systems where the physical world pushes back

Outcomes:

Reduced delays, errors, and lost context where physical reality could not be replayed.
Warehousing, excavation, and hardware work all became more observable, more reliable, and more actionable.

Bridged the gap between physical action and digital truth before the evidence disappeared.

References:
IoT, embedded, APIs, operational tooling
Warehouse tools, excavation records, prototype hardware

Systems under pressure.

Stabilized.

If you're building something like that,
I'd love to contribute.

tolacika.dev *gh/tolacika* *gh/tolacika*